

Experiment No. 4

Aim: Create an app to display weather using a public REST API.

Apparatus / Software Required :

Hardware	- Laptop(ASUS Vivobook 16x) - Minimum 8 GB RAM - AMD Ryzen 5 5600H with Radeon Graphics Processor - Internet Connection
Software	- Windows 11 Operating System - Android Studio (Quail) - Java Development Kit (JDK) - Android SDK - Android Emulator (Pixel 7) - Gradle Build System

Theory

A **REST API (Representational State Transfer Application Programming Interface)** is used to exchange data between an application and a server through HTTP requests.

In this experiment, an Android application is created to display weather information for a selected city. The user enters or selects a city name, and the application sends a request to a public weather REST API. The API returns weather data such as **city name, temperature, weather condition, feels-like temperature and humidity**.

The received data is processed in the Android application and displayed on the screen using different UI components. The application also provides city suggestions while entering the city name to reduce typing errors.

Implementation Steps:

1. Open Android Studio and create a new Android project.
2. Select Java as the programming language.
3. Design the weather screen using XML.
4. Add a search box for entering the city name.
5. Add city suggestions/autocomplete to help the user select the correct city.
6. Add a Check Weather button.
7. Create the required Java code to handle the button click event.
8. Connect the application with the public weather REST API.

9. Send the selected city name as a request to the API.
10. Receive the weather information from the API.
11. Extract the required values such as temperature, humidity and weather condition.
12. Display the received information on the weather card.
13. Run the application on the Pixel 7 Android Emulator.
14. Test the application with different cities such as Nagpur, Mumbai and Pune.
15. Verify that the correct weather information is displayed.

Files Used and Code Changes

File	Changes Made
<i>MainActivity.java</i>	Added API calling logic, button click handling, city search and displaying weather data.
<i>activity_main.xml</i>	Designed the main weather screen and added search field, button and weather information views.
<i>city_dropdown_item.xml</i>	Used for displaying city suggestions in the search/autocomplete list.
<i>AndroidManifest.xml</i>	Added the required Internet permission for accessing the REST API.
<i>build.gradle</i>	Added required dependencies/libraries used by the application.
<i>drawable - resources</i>	Added weather-related icons/backgrounds used in the interface.

Components Used

The following Android UI components were used:

1. **TextView** – to display headings, city name, temperature, humidity and weather condition.
2. **EditText / AutoCompleteTextView** – to enter and select the city name.
3. **Button** – to check the weather.
4. **ImageView** – to display weather icons.
5. **LinearLayout** – to arrange the UI components vertically.
6. **CardView / Weather Card** – to display weather details in a structured format.
7. **Toast** – to display messages or errors when required.

Important XML Attributes Used

XML Attribute	Purpose
android:layout_width	Defines the width of a component.
android:layout_height	Defines the height of a component.
android:text	Sets the text displayed by a component.
android:textSize	Sets the size of text.
android:textColor	Sets the color of text.
android:background	Sets the background of a component/layout.
android:padding	Adds space inside a component.
android:layout_margin	Adds space outside a component.
android:hint	Displays a hint inside the search field.
android:gravity	Controls the position/alignment of content.
android:orientation	Defines the direction of a LinearLayout.
android:src	Sets an image for an ImageView.

Important Java Methods Used

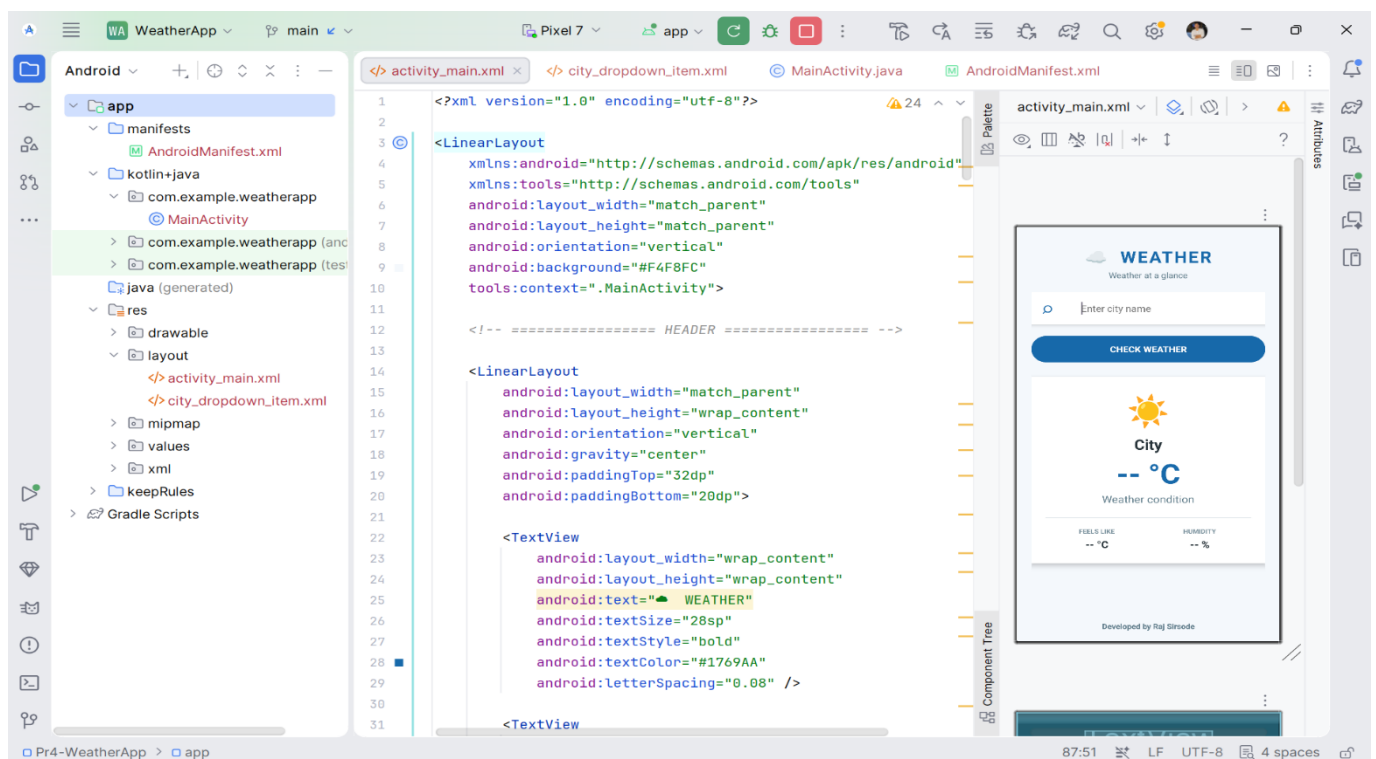
1. **onCreate()** – initializes the activity and connects Java code with the XML layout.
2. **findViewById()** – accesses UI components from the XML layout.
3. **setOnClickListener()** – handles the click event of the Check Weather button.
4. **getText()** – gets the city name entered by the user.
5. **trim()** – removes unnecessary spaces from the entered city name.
6. **API request method** – sends the weather request to the REST API.
7. **JSON parsing methods** – extract temperature, humidity, city and weather condition from the API response.
8. **setText()** – displays the received weather information on the screen.
9. **Toast.makeText()** – displays error or information messages.

Errors Faced and Troubleshooting

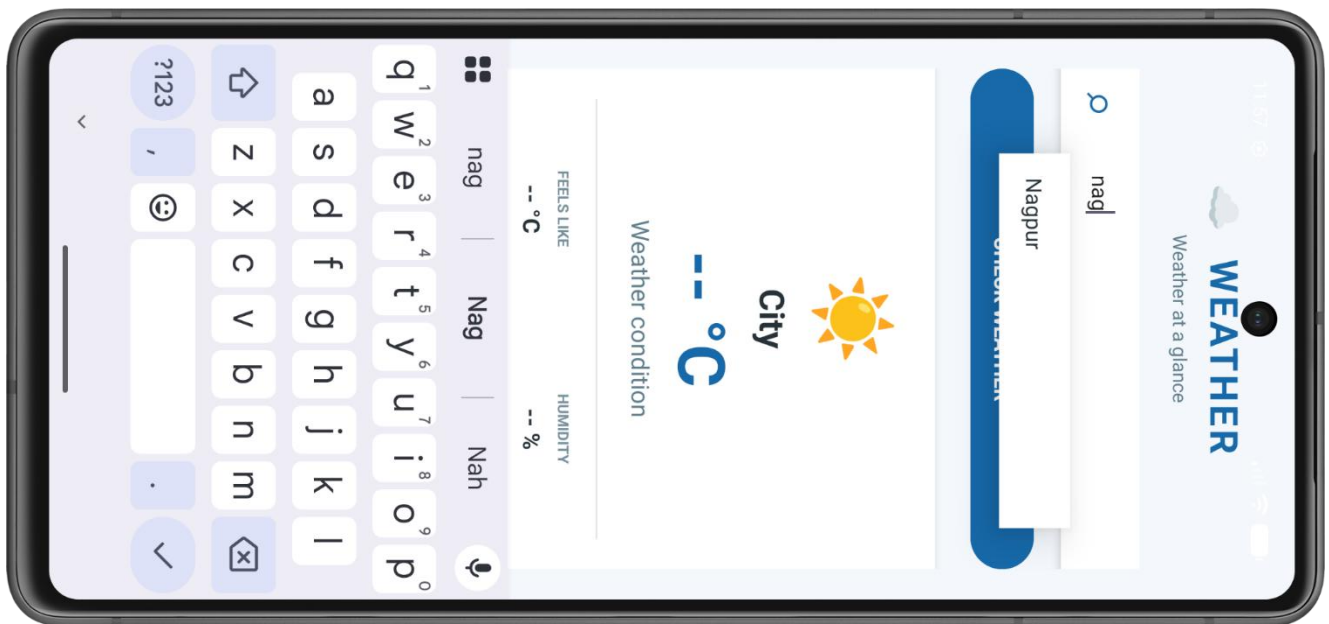
Error / Problem	Troubleshooting
Gradle project sync failed	Checked Gradle version and allowed the required Gradle files/dependencies to download.
Design editor unavailable	Successfully synced the Gradle project before using the Design editor.
Internet/API connection problem	Checked the internet connection and API request.
Weather data not displayed	Checked the API response and JSON parsing code.
Invalid city name	Added city suggestions/autocomplete to help select the correct city.
Typing mistakes in city name	Used city suggestions so the user can select the correct city.
API request failure	Checked the API URL, parameters and network connection.
UI alignment problems	Adjusted XML layout attributes such as margins, padding and gravity.

Output:

1. Weather app home screen



2. City suggestion while typing



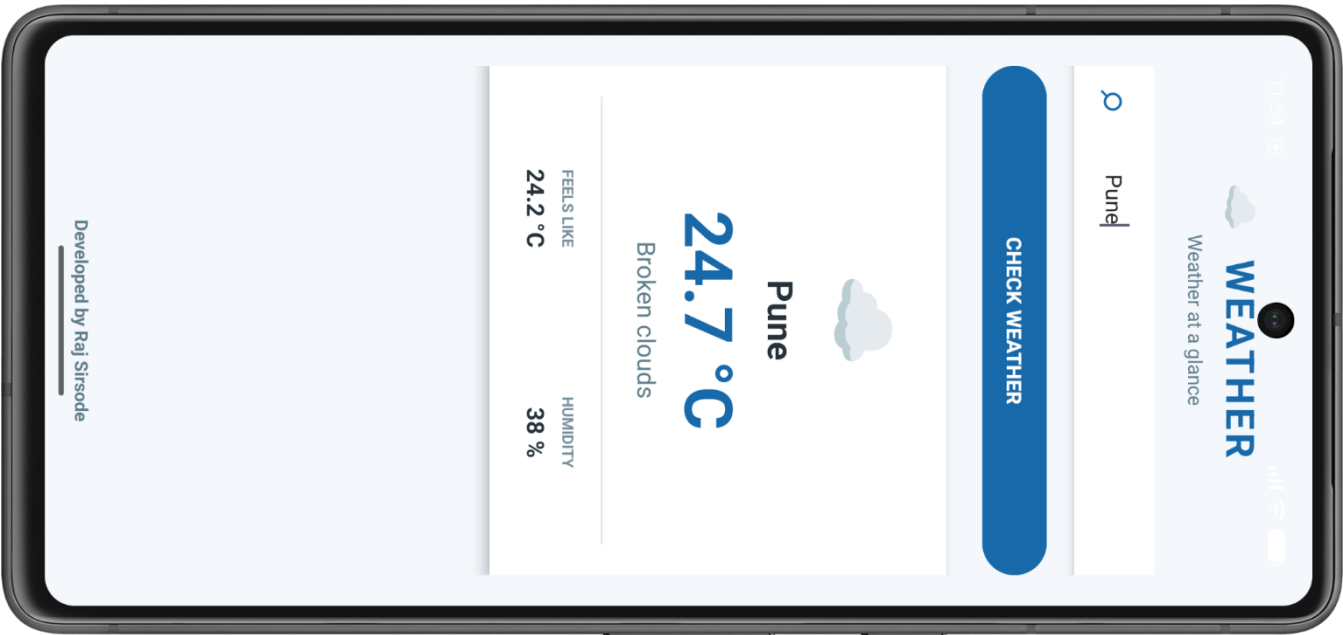
3. Nagpur weather result



4. Mumbai weather result



5. Pune weather result



Conclusion

The Weather Application was successfully created using Android Studio and Java. The application uses a public REST API to obtain real-time weather information for a selected city. The received data is displayed in the application in the form of temperature, weather condition, feels-like temperature and humidity. The application was tested on the Pixel 7 emulator with different cities such as Nagpur, Mumbai and Pune, and the weather information was displayed successfully. The experiment helped in understanding REST API integration, JSON data handling, Android UI components, event handling and network-based application development.

C (3)	A (3)	P (3)	T (9)	Sign